UNIVERSIDAD MAYOR
para espíritus emprendedores

AgroColab

Hito 2: El Motor que Sostiene la Promesa

Arquitectura de Desarrollo (Móvil y Web) · INGT1003

Integrantes: Matias Gonzalez Paredes
Joaquin Alveal Santos
Nicolas Reyes Orellana

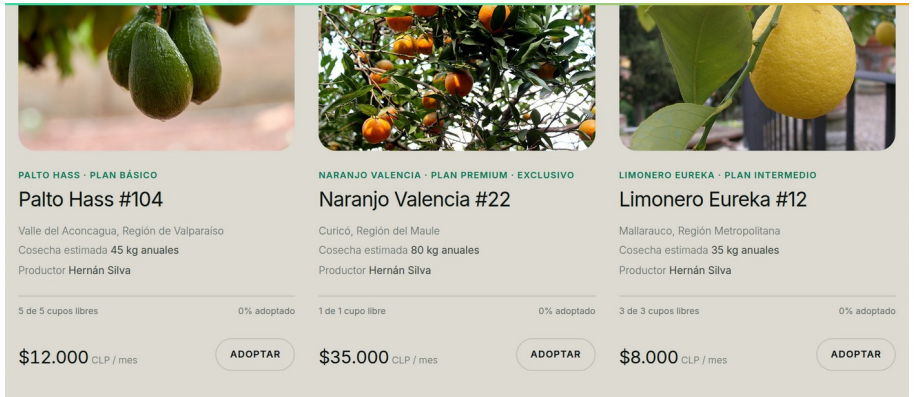


UNIVERSIDAD MAYOR
para espíritus emprendedores

El Producto y el Motor

El Producto: Plataforma Agro-Colab

Quien vive en la ciudad adopta un árbol o contrata una canasta, y su pago financia la cosecha.





El Motor: Lo que Hace el Servidor

Cuentas seguras

Verifica por correo, firma cada sesión y deja a cada persona ver solo lo suyo.

Árboles y cupos

Publica desde un catálogo cerrado y controla cuántos cupos quedan.

Pagos y pedidos

El monto lo fija el servidor. Sin pago aprobado no hay adopción.

Datos que permanecen

Todo queda en PostgreSQL y sobrevive a reinicios y despliegues.

UNIVERSIDAD MAYOR
para espíritus emprendedores

Demostración en Vivo



Peticiones reales contra el servicio. Seguimos un pedido durante toda su vida, mirando el código de estado y el dato guardado.

CREAR	LEER	ACTUALIZAR	LEER	BORRAR	LEER
201	200	200	200	200	404
POST Registra el pedido con su pago	GET El dato quedó guardado	PUT Cambia la comuna	GET Se ve el cambio	DELETE Cancela el plan	GET Ya no existe

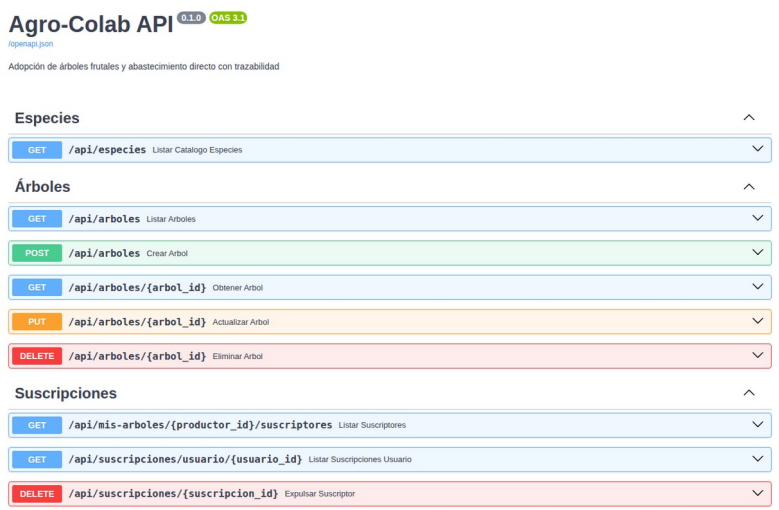
Y cuando algo falla, la API lo dice con su código

401	402	403	409	422
Sin token	Tarjeta sin fondos	Sin permiso	Plan sin pagar	Dato inválido

El Contrato de la API REST

Recursos, verbos y códigos

34 operaciones sobre árboles, pedidos, pagos y cuentas. Cada ruta usa un sustantivo y un verbo de HTTP.



Documentada en docs/API.md y en /docs

Medimos 29 peticiones reales

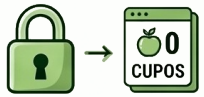


10 de 17 errores respondían 200

La interfaz lo ocultaba. Ahora el negocio lanza el error y main.py lo traduce a su código.

UNIVERSIDAD MAYOR
para espíritus emprendedores

Del Hito 1 al Hito 2: Qué Cambió

Lo que prometimos		Lo que construimos	Por qué cambió
	Suscribirse a un árbol	→ **Adoptar exige pagar: pago y cupo van juntos**	Se podía adoptar sin pagar
	Base de datos relacional	→ **PostgreSQL en producción, SQLite en local**	SQLite perdía datos al desplegar
	Verificar al usuario	→ **Tokens firmados y permisos por recurso**	Se confiaba en el id enviado
	Controlar los cupos	→ **Reserva atómica con UPDATE condicional**	Dos compras tomaban el mismo cupo

UNIVERSIDAD MAYOR
para espíritus emprendedores

Tecnologías y Trade-off

DECISIÓN: Python + FastAPI

FastAPI

con PostgreSQL como motor de datos

Curva de aprendizaje
Sintaxis accesible

Validación estricta
Con Pydantic



Documentación
/docs se genera sola

Python frente a Node.js
Más accesible para el equipo.
Costo: Dos lenguajes en el proyecto.

FastAPI frente a Express y Django
Valida y documenta; divide por dominio.
Costo: Integramos ORM y autenticación.

PostgreSQL frente a SQLite
Persiste y maneja escrituras concurrentes.
Costo: Servicio externo y dos motores que probar.

Lo que corregimos: el Hito 1 justificó FastAPI por su rendimiento asíncrono, pero ninguna ruta lo usa. Lo que la sostiene es la validación.

UNIVERSIDAD MAYOR
para espíritus emprendedores

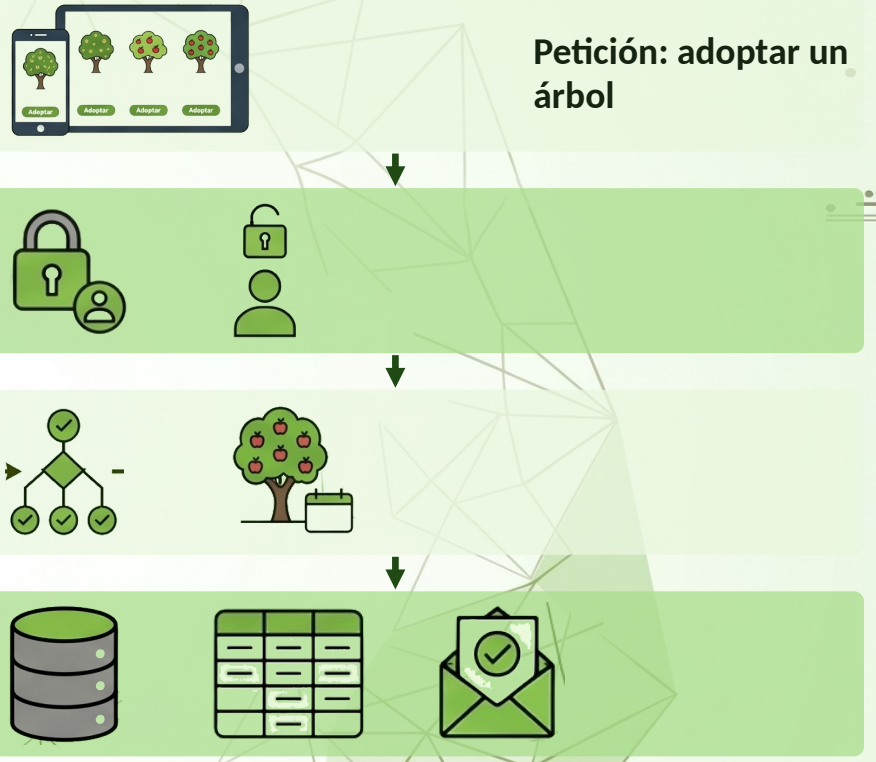
Arquitectura por Capas

Capa de Presentación frontend/js/api.js
Muestra el catálogo y envía la petición con el token.

Capa HTTP routers/ y main.py
Valida la entrada, llama al servicio y traduce los errores.

Capa de Negocio services/
Aplica las reglas: rol, monto del servidor, tarjeta y cupo.

Capa de Datos repositories/ y database.py
Reserva el cupo y confirma todo en una transacción.



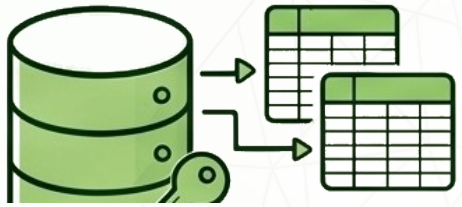
Petición: adoptar un árbol

89 → 0
consultas en routers

test_arquitectura.py: si una consulta sale de su capa, la integración continua falla.

UNIVERSIDAD MAYOR
para espíritus emprendedores

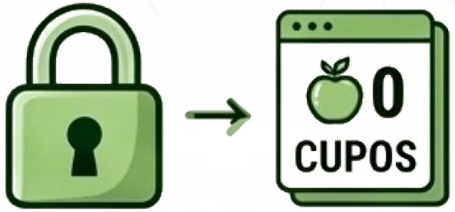
Persistencia y Último Cupo



El modelo

7 tablas con claves y relaciones. Una suscripción une a un usuario con un árbol.

La carpeta db/ trae el diagrama y el script que crea la base desde cero.



Dos compras, un cupo

1. A toma el cupo y bloquea la fila
2. B espera a que A termine
3. B reevalúa: ya no queda cupo
4. B no se cobra: responde 409

500

compras simultáneas en PostgreSQL

20 de 20 rondas con una sola aprobada

Credenciales

Fuera del código: variables de entorno, con .env.example como plantilla.

UNIVERSIDAD MAYOR
para espíritus emprendedores

Monolito o Distribuido

DECISIÓN:

Monolito modular



Costo que asumimos

Un error grave detiene todo, y no se puede escalar una parte por separado.

	Con monolito	Con servicios
Carga Decenas de usuarios; 100+ lecturas/s.	Un proceso alcanza	Nada que escalar aparte
Equipo 3 integrantes y un semestre.	Un repositorio y una suite	Demasiados servicios
Costo Plan gratuito: se detiene a los 15 min.	Un servicio y una base	Esperas en cadena
Despliegue Pago y cupo son una sola operación.	Una transacción local	Transacción distribuida

La revisaríamos con: pagos reales, correo que retrase el registro, equipos separados o una parte saturada.

UNIVERSIDAD MAYOR
para espíritus emprendedores

Conclusión

LOGROS HITO 2 - EL MOTOR

API REST con códigos correctos

Capas separadas y comprobadas

Datos en PostgreSQL

Sin vender dos veces un cupo



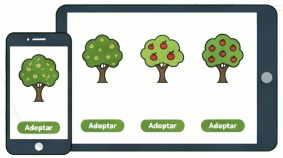
DESAFÍOS HITO 3 - EL PRODUCTO

Flujo completo para el docente

Manual de uso e instalación

Video de 2 minutos

Migraciones versionadas



MAYOR APRENDIZAJE - LO QUE NO SE VE EN PANTALLA



Un cupo vendido dos veces, datos perdidos al desplegar y errores que respondían 200.

Medir encontró lo que la interfaz ocultaba, y las pruebas hicieron seguro corregirlo.